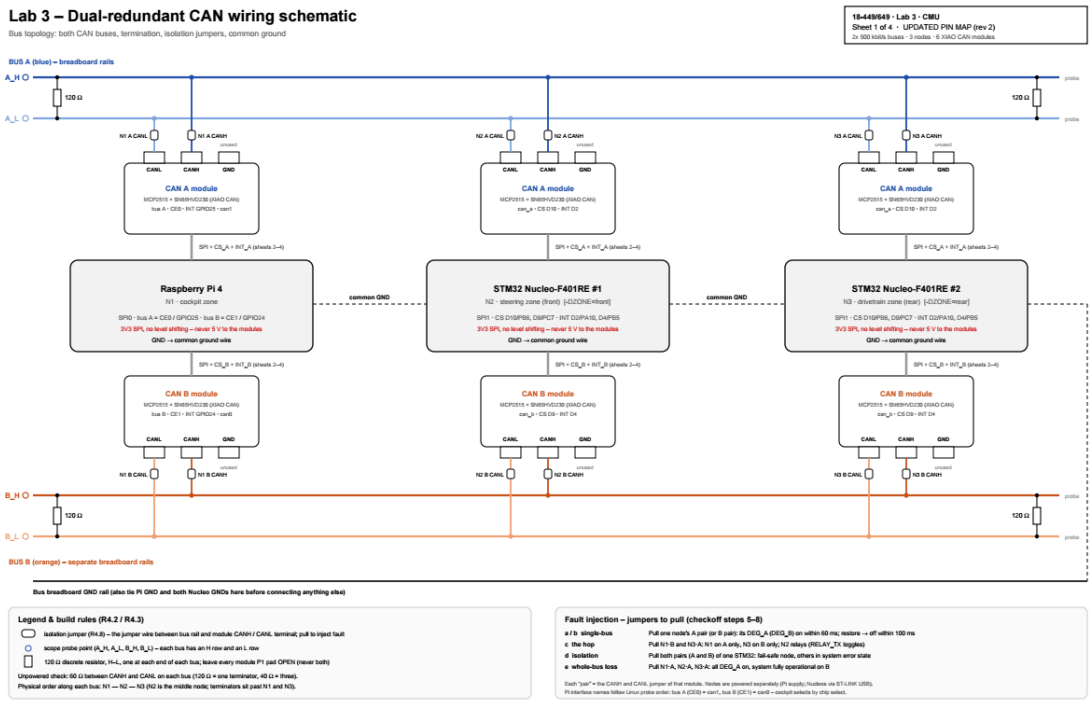
Lab 3 Report

1. Wiring schematic of the dual bus

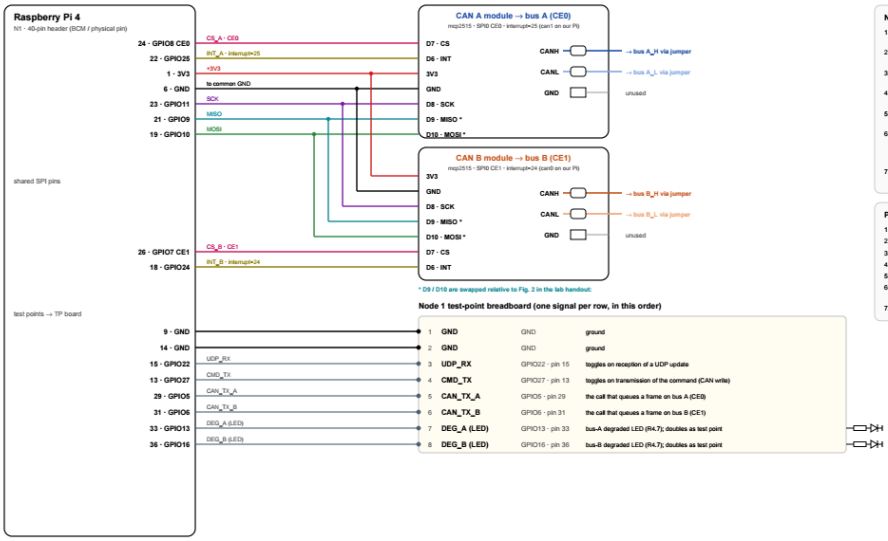


Lab 3 – Dual-redundant CAN wiring schematic

N1 · Raspberry Pi 4 (cockpit zone): SPI0 to two MCP2515 modules + test-point board

18-449/649 · Lab 3 · CMU
Sheet 2 of 4 · UPDATED PIN MAP (rev 2)
2x 500 kbit/s buses · 3 nodes · 6 XIAO CAN modules

Wire key: +3V3 · GND · SCK · MISO · MOSI · CS · INT (active low) · CANH/CANL → bus · test point



* D9 / D10 are swapped relative to Fig. 2 in the lab handout:

Node 1 test-point breadboard (one signal per row, in this order)

1	**GND**	GND	ground
2	**GND**	GND	ground
3	**UDP_RX**	GPIO22 · pin 15	toggles on reception of a UDP update
4	**CMD_TX**	GPIO27 · pin 13	toggles on transmission of the command (CAN write)
5	**CAN_TX_A**	GPIO5 · pin 29	the call that queues a frame on bus A (CE0)
6	**CAN_TX_B**	GPIO6 · pin 31	the call that queues a frame on bus B (CE1)
7	**DEG_A (LED)**	GPIO13 · pin 33	bus-A degraded LED (R4.7); doubles as test point
8	**DEG_B (LED)**	GPIO16 · pin 36	bus-B degraded LED (R4.7); doubles as test point

Notes

1. Pi is 3.3 V logic: modules connect straight to the 40-pin header, no level shifting. Leave module 5V unconnected.
2. SCLK, MOSI, MISO are shared by both modules; CE0/CE1 and GPIO25/GPIO24 are per-module.
3. Module D9/D10 are swapped vs handout Fig. 2: D9 = MISO → pin 21, D10 = MOSI → pin 19.
4. Power each pair of modules from the Pi 3V3 pin only; share GND only – never tie two boards' 3V3 together.
5. N1 has no CAN_RX test points (rows 3–4 are UDP_RX and CMD_TX). LED rows: LED + series resistor (~330 Ω) from the row to GND.
6. Pi config (/boot/firmware/config.txt): dtparam=spi=on · dtoverlay=mcp2515-can0,oscillator=16000000,interrupt=25 · dtoverlay=mcp2515-can1,oscillator=16000000,interrupt=24. Interrupt= must match the INT wires.
7. Bring-up: ip link set canX up type can bitrate 500000 (./pi/setup.sh); expect ERROR-ACTIVE. On our Pi can1 = bus A, can0 = bus B.

Pin changes vs previous schematic

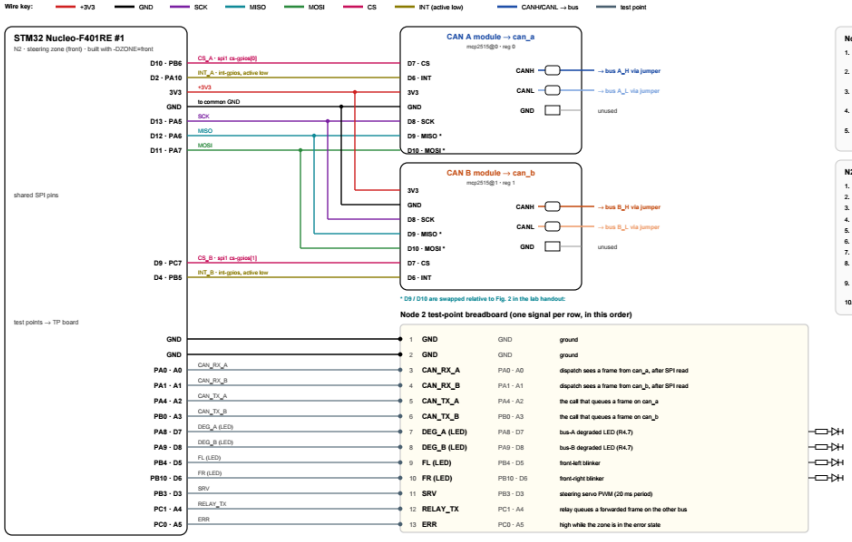
1. UDP_RX: GPIO5 (pin 29) → GPIO22 (pin 15)
2. CMD_TX: GPIO6 (pin 31) → GPIO27 (pin 13)
3. CAN_TX_A: GPIO13 (pin 33) → GPIO5 (pin 29)
4. CAN_TX_B: GPIO19 (pin 35) → GPIO6 (pin 31)
5. DEG_A LED: GPIO26 (pin 37) → GPIO13 (pin 33)
6. Unchanged: SPI0 pins, CE0/CE1, INT 25/24, DEG_B LED (GPIO16, pin 36).
7. Source: Lab3_Setup_and_Debug, section 2 (Raspberry Pi table).

Lab 3 – Dual-redundant CAN wiring schematic

N2 · STM32 Nucleo-F401RE #1 (steering zone): SPI1 to two MCP2515 modules + test-point board

18-449/649 · Lab 3 · CMU
Sheet 3 of 4 · UPDATED PIN MAP (rev 2)
2x 500 kbit/s buses · 3 nodes · 6 XIAO CAN modules

Wire key: +3V3 · GND · SCK · MISO · MOSI · CS · INT (active low) · CANH/CANL → bus · test point



* D9 / D10 are swapped relative to Fig. 2 in the lab handout:

Node 2 test-point breadboard (one signal per row, in this order)

1	**GND**	GND	ground
2	**GND**	GND	ground
3	**CAN_RX_A**	PA0 · A0	dispatch sees a frame from can_a, after SPI read
4	**CAN_RX_B**	PA1 · A1	dispatch sees a frame from can_b, after SPI read
5	**CAN_TX_A**	PA4 · A2	the call that queues a frame on can_a
6	**CAN_TX_B**	PB0 · A3	the call that queues a frame on can_b
7	**DEG_A (LED)**	PA8 · D7	bus-A degraded LED (R4.7)
8	**DEG_B (LED)**	PA9 · D8	bus-B degraded LED (R4.7)
9	**FL (LED)**	PB4 · D5	front-left blinker
10	**FR (LED)**	PB10 · D6	front-right blinker
11	**SRV**	PB3 · D3	steering servo PWM (20 ms period)
12	**RELAY_TX**	PC1 · A4	relay queues a forwarded frame on the other bus
13	**ERR**	PC0 · A5	high while the zone is in the error state

Notes

1. Both modules sit on SPI1 (1 MHz, osc-freq 16 MHz, bitrate 500 kbit/s per overlay). SCK/MISO/MOSI are shared; CS and INT are per-module.
2. Module D9/D10 are swapped vs handout Fig. 2: D9 = MISO → D12/PA6, D10 = MOSI → D11/PA7.
3. PA5 (SCK, D13) is also the Nucleo user LED LD2 – expect it to blink with SPI traffic.
4. Power the modules from the Nucleo 3V3 pin only (never 5 V); leave module 5V unconnected. PA13/PA14 stay free for SWD.
5. All pins are on the Arduino header (unmarked). LED rows: LED + series resistor (~330 Ω, suggested) from the row to GND.

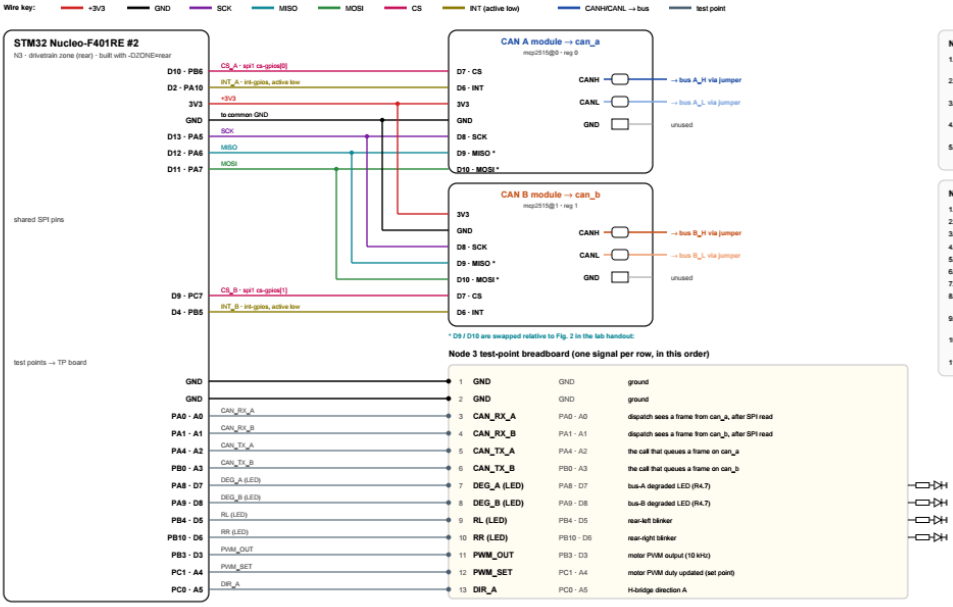
N2 pin changes vs previous schematic

1. CAN_RX_A/B: PB8/PB9 (D15/D14) → PA0/PA1 (A0/A1)
2. CAN_TX_A/B: PA0/PA1 → PA4/PB0 (A2/A3)
3. DEG_A/B LED: PC15/PC12 (M) → PA8/PA9 (D7/D8)
4. FL/FR: PD2/PC11 (M) → PB4/PB10 (D5/D6)
5. SRV: PA8 (D7) → PB3 (D3)
6. RELAY_TX: PC8 (M) → PC1 (A4)
7. ERR: PA15 (M) → PC0 (A5)
8. All test points are now on the Arduino header – no morpho wires needed.
9. Unchanged: SPI1 pins, CS (D10/D9), INT (D2/D4). Test button B1 (blue) = PC13, on board.
10. Source: Lab3_Setup_and_Debug, section 2.

Lab 3 – Dual-redundant CAN wiring schematic

N3 · STM32 Nucleo-F401RE #2 (drivetrain zone): SPI1 to two MCP2515 modules + test-point board

18-449/649 · Lab 3 · CMU
Sheet 4 of 4 · UPDATED PIN MAP (rev 2)
2x 500 kbit/s buses · 3 nodes · 6 XIAO CAN modules



Node 3 test-point breadboard (one signal per row, in this order)

1	GND	GND		ground
2	GND	GND		ground
3	CAN_RX_A	PA0 · A0		dispatch sees a frame from can_a, after SPI read
4	CAN_RX_B	PA1 · A1		dispatch sees a frame from can_b, after SPI read
5	CAN_TX_A	PA4 · A2		the call that queues a frame on can_a
6	CAN_TX_B	PB0 · A3		the call that queues a frame on can_b
7	DEG_A (LED)	PA8 · D7		bus-A degraded LED (R4.7)
8	DEG_B (LED)	PA9 · D8		bus-B degraded LED (R4.7)
9	RL (LED)	PB4 · D5		rear-left blinker
10	RR (LED)	PB10 · D6		rear-right blinker
11	PWM_OUT	PB3 · D3		motor PWM output (10 kHz)
12	PWM_SET	PC1 · A4		motor PWM duty updated (set point)
13	DIR_A	PC0 · A5		H-bridge direction A

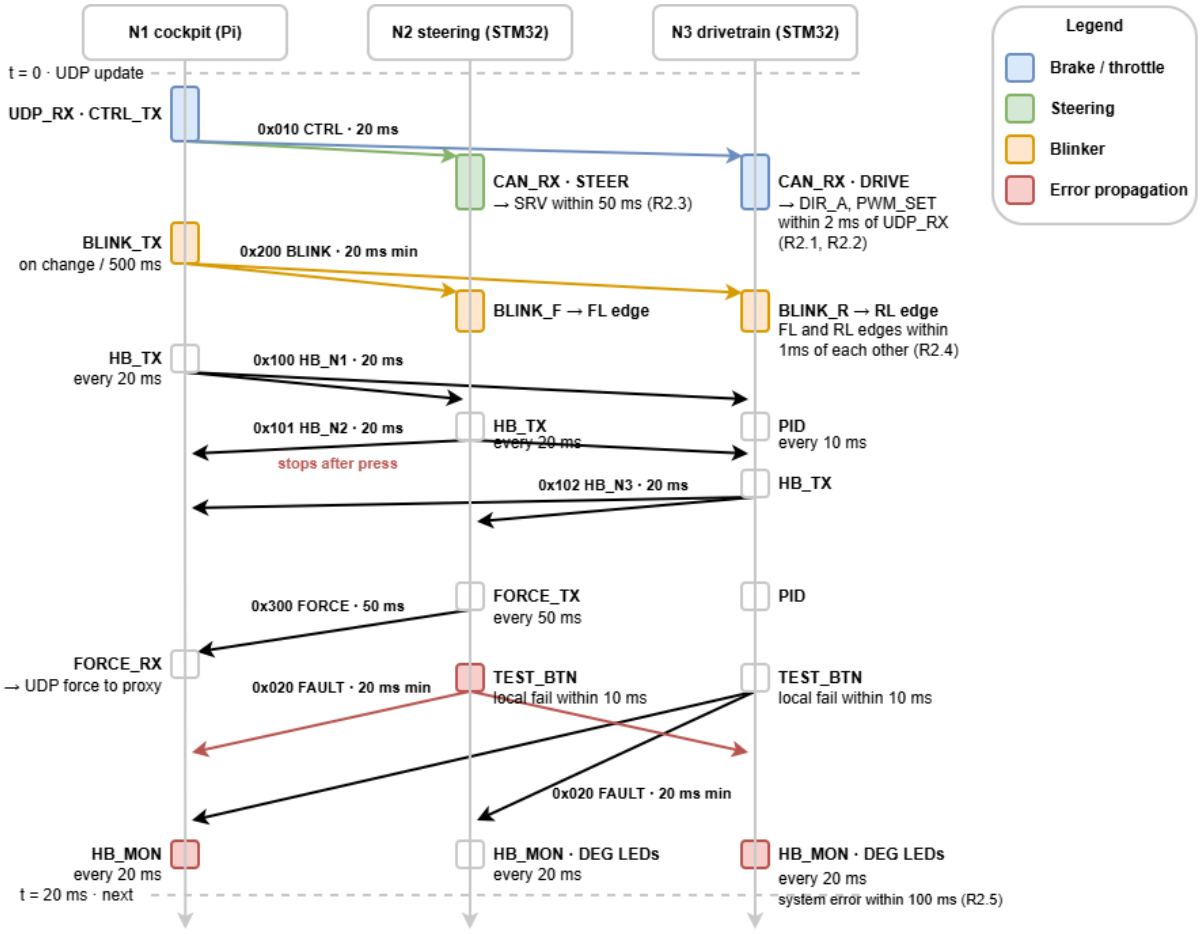
Notes

1. Both modules sit on SPI1 (1 MHz, osc-freq 16 MHz, bitrate 500 kbit/s per overlay). SCK/MISO/MOSI are shared; CS and INT are per-module.
2. Module D9/D10 are swapped vs handout Fig. 2: D9 = MISO → D12/PA6, D10 = MOSI → D11/PA7.
3. PA5 (SCK, D13) is also the Nucleo user LED LD2 – expect it to blink with SPI traffic.
4. Power the modules from the Nucleo 3V3 pin only (never 5 V); leave module 5V unconnected. PA13/PA14 stay free for SWD.
5. All pins are on the Arduino header (unmarked). LED rows: LED + series resistor (~330 Ω, suggested) from the row to GND.

N3 pin changes vs previous schematic

1. CAN_RX_A/B: PC0/PC1 (A5/A4) → PA0/PA1 (A0/A1)
2. CAN_TX_A/B: PC2/PC3 (MI) → PA4/PB0 (A2/A3)
3. DEG_A/B LED: PC10/PC12 (MI) → PA8/PA9 (D7/D8)
4. RL/RR: PC8 (MI) / PB0 (A3) → PB4/PB10 (D5/D6)
5. PWM_OUT: PA8 (D7) → PB3 (D3)
6. PWM_SET: PC11 (MI) → PC1 (A4)
7. DIR_A: PC9 (MI) → PC0 (A5)
8. All test points are now on the Arduino header – no morpho wires needed.
9. Unchanged: SPI1 pins, CS (D10/D9), INT (D2/D4). Test button B1 (blue) = PC13, on board.
10. Check: DIR_B (second H-bridge, was PB12) is not in the Setup & Debug pin table – confirm it in the firmware overlay before wiring.
11. Source: Lab3_Setup_and_Debug, section 2.

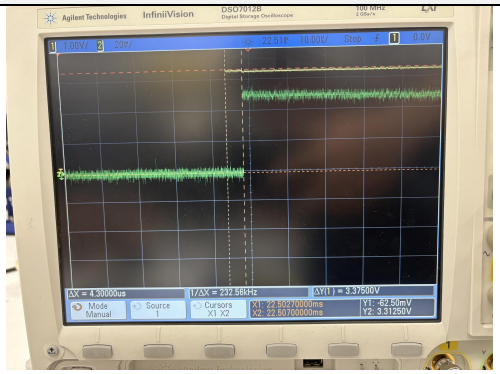
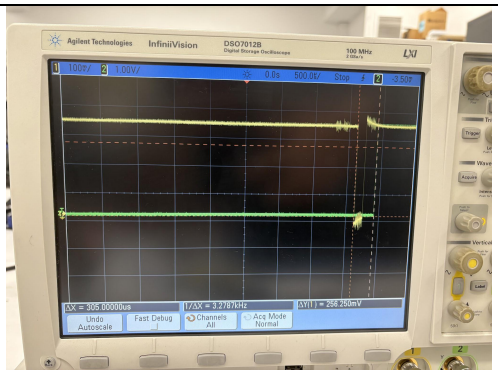
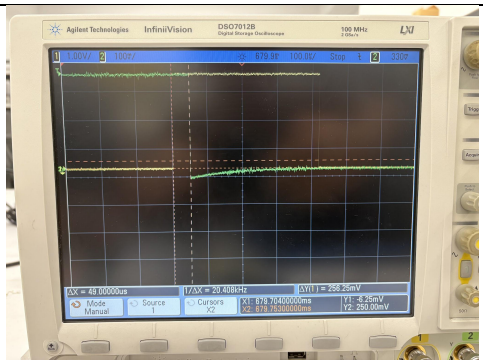
2. Task communication graph figure

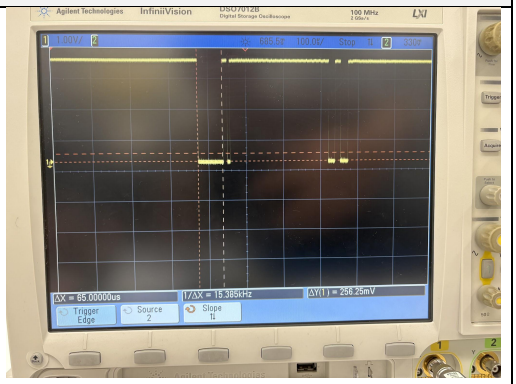


Not to scale. Each frame is sent on both buses; the receiver keeps the first copy.

3. Single-bus analysis

Measurements for this part (4 of 6)

#	Number	Start probe	Stop probe	Your max (µs)	Screenshot
1	Pi stage (N1)	UDP_RX, toggled right after the socket read returns	CMD_TX, toggled right after the first CAN write returns	4.3	
2	CAN_RX C (N3)	INT line of the CAN A module falling	CAN_RX_A, toggled when dispatch first gets the frame	305	
3	DRIVE C (N3)	CAN_RX_A	DIR_A changing on a brake frame	49	

#	Number	Start probe	Stop probe	Your max (µs)	Screenshot
4	SPI frame load (B) (N3)	Module CS line going low	CS going high again (longest low window you see)	65	

Pi toggle language (C or Python): C

Pi process pinned / SCHED_FIFO: SCHED_FIFO priority 40, not pinned

For each WCET and latency you use: measured or assumed, and how it was measured: Measured from RPi and STM test points, all other WCETs assumed, frame time computed.

Priorities are the Zephyr priorities in our code (lower = higher); CAN_RX is the driver thread (2) plus dispatch (4). Equal-priority tasks count as interference. B = 0 for the lowest-priority tasks. Every ⌈R/Tj⌉ = 1, so R = C + B + (sum of hp C).

Node RTA table: N3

Task	Priority	C (µs)	T	D	B (µs)	R (µs)	R ≤ D?
CAN_RX	2 / 4	305	20 ms per message, 7 per bus	0.3 ms	65	305 + 65 = 370	No
DRIVE	5	49	20 ms	0.5 ms	65	49 + 65 + (305 + 20) = 439	Y
BLINK_R	5	20	20 ms	0.5 ms	65	20 + 65 + (305 + 49) = 439	Y
TEST_BTN	6	20	50 ms	10 ms	65	20 + 65 + (305 + 49 + 20 + 200) = 659	Y
PID	6	200	10 ms	10 ms	65	200 + 65 + (305 + 49 + 20 + 20) = 659	Y
HB_TX	7	120	20 ms	20 ms	0	120 + 0 + (305 + 49 + 20 + 20 + 200 + 30) = 744	Y
HB_MON	7	30	20 ms	20 ms	0	30 + 0 + (305 + 49 + 20 + 20 + 200 + 120) = 744	Y

Node RTA table: N2

Task	Priority	C (µs)	T	D	B (µs)	R (µs)	R ≤ D?
CAN_RX	2 / 4	80	20 ms per message, 7 per bus	0.3 ms	65	80 + 65 = 145	Y
STEER	5	50	20 ms	1 ms	65	50 + 65 + (80 + 20) = 215	Y
BLINK_F	5	20	20 ms	0.5 ms	65	20 + 65 + (80 + 50) = 215	Y
TEST_BTN	6	20	50 ms	10 ms	65	20 + 65 + (80 + 50 + 20) = 235	Y
FORCE_TX	8	120	50 ms	50 ms	0	120 + 0 + (80 + 50 + 20 + 20 + 120 + 30) = 440	Y
HB_TX	7	120	20 ms	20 ms	65	120 + 65 + (80 + 50 + 20 + 20 + 30) = 385	Y
HB_MON	7	30	20 ms	20 ms	65	30 + 65 + (80 + 50 + 20 + 20 + 120) = 385	Y

Message RTA table

Message	J (µs)	C (µs)	B (µs)	w (µs)	R (µs)
0x010 CTRL	4.3	270	270	270	4.3 + 270 + 270 = 544.3

End-to-end chain table

Chain	Stages to add up	Total (us)	Deadline (us)	Met?
Brake / throttle	Pi + CTRL on bus + CAN_RX (N3) + DRIVE 4.3 + 540 + 370 + 439	1,353.3	2,000	Y
Steering	Pi + CTRL on bus + CAN_RX (N2) + STEER + one servo period 4.3 + 540 + 145 + 215 + 20,000	20,904.3	50,000	Y
Blinker start	Pi + BLINK on bus + CAN_RX (N3) + BLINK_R 4.3 + 1,890 + 370 + 439	2,703.3	100,000	Y
Blinker sync	N3 receive path minus N2 receive path, worst case	709	1,000	Y

Chain	Stages to add up	Total (us)	Deadline (us)	Met?
	(370 + 439) − (80 + 20)			
Error propagation	TEST_BTN (N2) + 3 missed heartbeats (66 ms) + one HB_MON period (20 ms) + HB_MON (N3) 235 + 66,000 + 20,000 + 744	86,979	100,000	Y

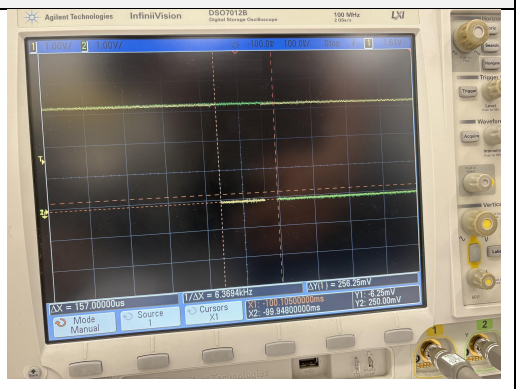
BLINK on bus: 0x200 waits behind CTRL, FAULT and three heartbeats, so w = 270 + 5 × 270 = 1,620 and w + C = 1,890 µs.

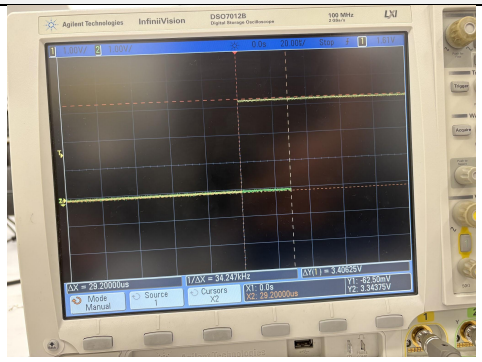
N3 CAN_RX misses 0.3 ms because its measured C (305 µs) is already over it: both bus copies of a frame arrive together on one SPI bus, so the bus A copy can wait for the bus B read. The brake chain still meets 2 ms. Fix: handle the frame in the driver's receive callback instead of a second thread.

4. Dual-bus and relay analysis

Add redundancy and analyze one relay case: **N1 off bus B, N3 off bus A, N2 relays** (scenario c). N1 can only talk on A, N3 can only hear on B, so N2 relays A to B (and HB_N3 the other way, B to A). The relay always runs on an STM32; a relay through N1 is not tested because Linux has no worst-case bound.

Measurements for this part (2 of 6)

#	Number	Node	Start probe	Stop probe	Your max (µs)	Screenshot
5	CAN_RX C	N2	INT line of the CAN A module falling	CAN_RX_ A on N2	157	

#	Number	Node	Start probe	Stop probe	Your max (µs)	Screenshot
6	RELAY C	N2	CAN_RX_A on N2, with N1 off B and N3 off A	RELAY_TX, toggled after the send on bus B returns	29.2	

Relay design: defend your choices

Question	Your answer
Who relays, and when? Only N2, or every node on both buses? What is the bus load if everything is always relayed?	Both STM32s: N2 here, N3 when N1 and N2 are split. A node relays a zone's frames onto the other bus once it hasn't heard that zone's heartbeat directly there for 30 ms, if that bus works for the node. If everything were always relayed, each bus would carry about 22.6 % instead of 8.6 %.
How do you avoid loops? How does a node tell a relayed copy from a duplicate it should drop? Name the sequence counter / per-bus "already seen" check and the state it needs.	A relayed copy keeps its ID and seq and sets a relayed flag (byte 0, 0x10); flagged frames are never relayed again. Each receiver stores, per ID, the last seq and the buses it came on, and drops the same seq from the other bus. Relay decisions count only heartbeats heard directly, so two relays can't switch each other off.
Does the relay need to know the fault exists? Always-forward costs bandwidth but needs no detection; forward-on-detection is cheaper but adds the 60 ms detection time to the first relayed message. Which did you pick, and what does the brake chain look like during those 60 ms?	Forward-on-detection, to save bandwidth, with detection cut to 30 ms (one missed heartbeat). Until relaying starts, N3 gets no CTRL and holds its last command, so a brake can be delayed up to about 50 ms. No zone is declared failed in that window (65 ms timeout).
What priority does RELAY get on N2? Above STEER? Above the driver RX thread? What happens to N2's own deadlines?	Priority 3: below the driver RX thread (2), above CAN_RX dispatch (4) and STEER (5). N2's deadlines are still met (table below).

Re-analysis table: N2 with RELAY

Task	Priority	C (µs)	T	D	B (µs)	R (µs)	R ≤ D?
CAN_RX	2 / 4	157	20 ms per message, 7 per bus	0.3 ms	65	157 + 65 = 222	Y
RELAY	3	29.2	every frame N2 forwards	on the brake chain	65	29.2 + 65 + (157) = 251.2	See chain
STEER	5	50	20 ms	1 ms	65	50 + 65 + (157 + 29.2 + 20) = 321.2	Y
BLINK_F	5	20	20 ms	0.5 ms	65	20 + 65 + (157 + 29.2 + 50) = 321.2	Y
TEST_BTN	6	20	50 ms	10 ms	65	20 + 65 + (157 + 29.2 + 50 + 20) = 341.2	Y
FORCE_TX	8	120	50 ms	50 ms	0	120 + 0 + (157 + 29.2 + 50 + 20 + 20 + 120 + 30) = 546.2	Y
HB_TX	7	120	20 ms	20 ms	65	120 + 65 + (157 + 29.2 + 50 + 20 + 20 + 30) = 491.2	Y
HB_MON	7	30	20 ms	20 ms	65	30 + 65 + (157 + 29.2 + 50 + 20 + 20 + 120) = 491.2	Y

Bus utilization for your relay design

Relayed frame onto this bus	Period	Added utilization
CTRL, onto B	20 ms	1.35 %
BLINK, onto B	20 ms	1.35 %
HB_N1, onto B	20 ms	1.35 %
FAULT from N1, onto B	20 ms	1.35 %
HB_N3, onto A	20 ms	1.35 %
FAULT from N3, onto A	20 ms	1.35 %

Bus	New utilization with your relay	≤ 50 %?
A	8.6% + 2 * 1.35% = 11.3%	Y
B	8.6% + 4 * 1.35% = 14.0%	Y

Relayed CTRL on bus B

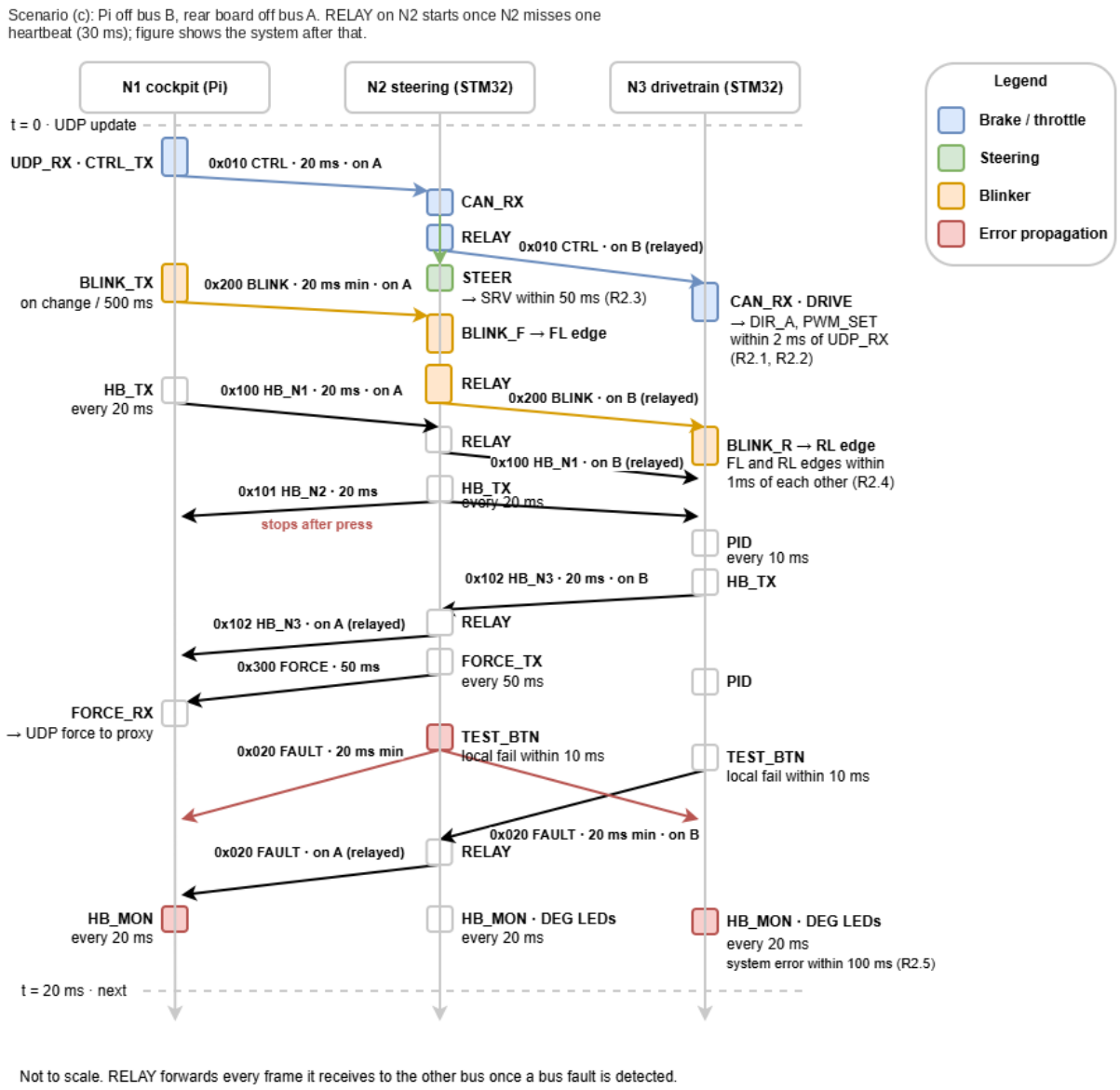
Message	J (µs)	C (µs)	B (µs)	w (µs)	R (µs)
0x010 CTRL, relayed	Pi (#1) + 540 + CAN_RX R (N2) + RELAY R (N2) 4.3 + 540 + 222 + 251.2 = 1,017.5	270	270	270	1,017.5 + 270 + 270 = 1,557.5

Chains in scenario (c): verdict and what changed

Chain	Stages to add up	Total (us)	Deadline (us)	Met ?	What changed
Brake / throttle	Relayed CTRL R + CAN_RX (N3) + DRIVE 1,557.5 + 370 + 439	2,366.5	2,000	N	Second hop: CAN_RX (N2), RELAY, CTRL on B
Steering	Pi + CTRL on A + CAN_RX (N2) + STEER + one servo period 4.3 + 540 + 222 + 321.2 + 20,000	21,087.5	50,000	Y	N2 times with RELAY
Blinker start	Pi + BLINK on A + CAN_RX (N2) + RELAY + BLINK on B + CAN_RX (N3) + BLINK_R 4.3 + 1,890 + 222 + 251.2 + 1,890 + 370 + 439	5,066.5	100,000	Y	Second hop to N3
Blinker sync	N3 receive path minus N2 receive path, worst case (222 + 251.2 + 1,890 + 370 + 439) − (157 + 20)	2,995.2	1,000	N	Second hop on N3's side only
Error propagation	TEST_BTN (N2) + 3 missed heartbeats (66 ms) + one HB_MON period (20 ms) + HB_MON (N3) 341.2 + 66,000 + 20,000 + 744	87,085.2	100,000	Y	N2 times with RELAY; no relay needed (N2 is on both buses)

Brake misses 2 ms by 0.37 ms because of the second hop. We report scenario (c) as a permitted degraded case (about 2.4 ms plus the Pi stage, shown by the DEG LEDs); meeting 2 ms would need forwarding inside N2's receive callback. Relaying also starts only 30 ms after the fault, so N3 holds its last command until then.

Blinker sync misses because only N3's copy takes the hop. Fix: N2 delays its own edge by the hop time when it relays BLINK; otherwise it is part of the same degraded case.



Not to scale. RELAY forwards every frame it receives to the other bus once a bus fault is detected.

5. Fault-injection matrix

Recovery is measured on the scope from restoring the jumper to the DEG (or ERR) test point going low.

Scenario	Jumpers Pulled	N1 State	N2 State	N3 State	DEG LEDs On	Measured Recovery Time (ms)
(a) one node off A	A pair of N1, N2 or N3	Degraded if pulled, else normal	Degraded if pulled, else normal	Degraded if pulled, else normal	DEG_A on the pulled node	2.3 ms (N1 pulled)
(b) one node off B	B pair of N1, N2 or N3	Degraded if pulled, else normal	Degraded if pulled, else normal	Degraded if pulled, else normal	DEG_B on the pulled node	3.4 ms (N2 pulled)
(c) the hop	N1 off B, N3 off A	Degraded (bus B)	Normal, relaying between A and B	Degraded (bus A)	DEG_B on N1, DEG_A on N3	2.3 ms
(d) N2 isolated	Both N2 pairs	System error, sends FAULT	Fail-safe: ERR high, hazards, servo held	System error: braked, hazards	DEG_A, DEG_B on N2	4 ms
(d) N3 isolated	Both N3 pairs	System error, sends FAULT	System error: ERR high, hazards, still steers	Fail-safe: braked, hazards	DEG_A, DEG_B on N3	2.9 ms
(d) N1 isolated	Both N1 pairs	Fail-safe	System error: ERR high, hazards, servo held	System error: braked, hazards	DEG_A, DEG_B on N1	5 ms
(e) whole-bus loss	Every node's A pair, then every node's B pair	Degraded (lost bus)	Degraded (lost bus)	Degraded (lost bus)	Lost bus's DEG on all three	A: 3.8 ms, B: 5.6 ms
(f) restore	Put back any pulled jumper	Normal	Normal	Normal	None	5.6 ms (worst above)

6. Team-defined values table and Lab 1 revision history

Value	Field / message	Valid min	Valid max	Units	Source of range
Steering	CTRL steer (int16)	-450	450	deg	Wheel is 900° lock to lock
Throttle	CTRL throttle (u8)	0	100	%	Pedal travel
Brake	CTRL brake (u8)	0	100	%	Pedal travel; anything above 0 brakes
Buttons	CTRL buttons (u8)	0x00	0x07	bits	Bit 0 left, 1 right, 2 test; 3-7 must be 0
Force feedback	FORCE (int8)	-100	100	-	Dummy counter this lab, no load mapping; Pi clamps to ±20
Turn signal cancel	CTRL steer	-150	150	deg	Turn signal cancels after the wheel passes this and comes back
Steering → servo	STEER	1000	2000	µs	Linear: -450 deg → 1000 µs, 450 deg → 2000 µs
Throttle → motor	DRIVE	0	100	% duty	Throttle % sets PWM duty directly (no encoders this lab)

Date	Version	Change to Lab 1 design
10/20/2026	1.1	Adopted the Lab 3 template message catalog and task set, added the CAN relay on both STM32s.